EE5203 L11 Lab Guide

I2C and SPI — find a device, drive a display, prove a bus - Basri Erdogan

Mission: Discover the LCD's address with a bus scan instead of guessing it, show live ADC readings on the display at a rate you can justify, make a NACK happen on purpose — and prove an SPI link works with a single loopback jumper.

Prepare before class

- ☐ Re-read the theory slides through "Loopback."
- ☐ Bring your working L10 project (tick + PWM + ADC + UART).
- ☐ Kit: 16×2 LCD with I2C backpack, breadboard, jumpers.
- ☐ Compute: your LCD's datasheet says `0x27`. What do you pass to HAL?

Learning objectives

By checkoff time, you can:

1. scan an I2C bus and interpret what answered;
2. convert between 7-bit and HAL 8-bit addresses in both directions;
3. drive a real I2C device and justify its refresh rate against a time budget;
4. configure SPI, verify `CPOL`/`CPHA` in `CR1`, and prove the link by loopback.

Hardware and files

Item	Required value
Board	Nucleo-F401RE — from a copy of the L10 project, saved as `L11_Buses`
LCD	16×2 with PCF8574 backpack: `VCC`→5V, `GND`→GND, `SDA`→PB9 (D14), `SCL`→PB8 (D15)
I2C1	100 kHz standard mode; pull-ups are on the backpack
SPI2	`SCK`=PB13, `MISO`=PB14, `MOSI`=PB15; loopback jumper **PB15 → PB14**
Knob	10 kΩ pot on PA0, from L09

The LCD backpack runs from **5 V** — its contrast will be unreadable on 3.3 V. Its `SDA`/`SCL` lines are still safe for the STM32 because they are open drain and pulled up to the backpack's own rail; do not add your own pull-ups to 5 V.

Configure in CubeMX

1. **Connectivity → I2C1**: Mode `I2C`, Speed `100 kHz`. Confirm PB8/PB9 (AF4).
2. **Connectivity → SPI2**: Mode `Full-Duplex Master`, Data Size `8 Bits`, `CPOL = Low`, `CPHA = 1 Edge` (mode 0), Prescaler 32.
3. Generate, then check `GPIOB->OTYPER` bits 8 and 9 in the generated init — they must be **open drain**.

Checkpoint A - Who is on the bus?

Do not type the address in from the datasheet. Find it:

```
for (uint8_t a = 1; a < 128; a++) {
    if (HAL_I2C_IsDeviceReady(&hi2c1, (uint16_t)(a << 1), 2, 5) == HAL_OK) {
        char b[32];
        int n = snprintf(b, sizeof b, "found 0x%02X\r\n", a);
        HAL_UART_Transmit(&huart2, (uint8_t *)b, (uint16_t)n, 100);
    }
}
```

Evidence for Checkpoint A: the scan reports one device over the L10 serial link. State its **7-bit** address, and the **8-bit** value you will pass to HAL. Then unplug `SDA` and re-run: the scan finds nothing. Both results are required.

Checkpoint B - The display works

Initialise the LCD in 4-bit mode and write a line. Each character costs four I2C bytes, so measure before you choose a refresh rate.

```
if ((g_ms - t_lcd) >= 500U) {          /* 2 Hz - justify this number */
    t_lcd = g_ms;
    uint16_t raw = adc_read(ADC_CHANNEL_0);
    uint32_t mv  = (raw * 3300u) / 4095u;
    char l[17];
```

```
    snprintf(l, sizeof l, "mV=%-4lu raw=%4u", (unsigned long)mv, (unsigned)raw);
    lcd_goto(0, 0);
    lcd_puts(l);
}
```

Evidence for Checkpoint B: live millivolts on the display, tracking the knob. Toggle a spare GPIO around the write and state how long a 16-character line actually takes. Compare that against your refresh period and **justify the rate you chose** — this is the assessed part, not the display itself.

Checkpoint C - Make it fail on purpose

```
HAL_StatusTypeDef st = HAL_I2C_Master_Transmit(&hi2c1, (0x55 << 1), &d, 1, 10);
/* nothing lives at 0x55 */
```

Evidence for Checkpoint C: `st` is `HAL_ERROR`, and `I2C1->SR1` shows the `AF` (acknowledge failure) bit set. Explain what the bus did, and why a UART could never report this.

Checkpoint D - SPI loopback

Fit one jumper from **PB15 (MOSI) to PB14 (MISO)**.

```
uint8_t tx = 0xA5, rx = 0x00;
HAL_SPI_TransmitReceive(&hspi2, &tx, &rx, 1, 100);   /* rx should be 0xA5 */
```

Evidence for Checkpoint D: `rx == 0xA5`. Then read `SPI2->CR1` and state your `CPOL` and `CPHA` bits and which mode they make. Remove the jumper and re-run: `rx` becomes `0x00` or `0xFF` with **no error reported** — which is the point of the exercise.

Individual extension

Pick one, predict first:

- **Fast mode:** reconfigure I2C to 400 kHz, re-measure the 16-character line, and compare against your 100 kHz figure. Does it scale as expected?
- **Mode mismatch:** set SPI to mode 3 with the loopback still fitted. Predict what `rx` becomes before you look, then explain the result.
- **Two-line display:** show millivolts on line 1 and the PWM duty on line 2, updating only the characters that changed. Measure the saving.

Acceptance checklist

- ☐ `.ioc`: I2C1 at 100 kHz on PB8/PB9; SPI2 full-duplex master mode 0.
- ☐ `OTYPER` bits 8/9 shown as open drain.
- ☐ Bus scan finds the LCD; 7-bit and 8-bit addresses both stated correctly.
- ☐ Display shows live data; refresh rate measured **and justified**.
- ☐ `AF` demonstrated deliberately and explained.
- ☐ SPI loopback returns the sent byte; `CPOL`/`CPHA` read from `CR1`.
- ☐ Jumper removed: silent wrong data, and the student can say why.

Individual oral check

- Your datasheet says `0x27`. What do you pass to HAL, and why?
- What holds `SDA` high, and what happens without it?
- How long does one 16-character line take at 100 kHz? Show the arithmetic.
- Which flag reports a NACK, and what does a NACK actually mean?
- Why does SPI not have an equivalent flag?
- Six identical sensors, limited pins — which bus, and why?

Submit

Submit `main.c`, the bus-scan output, and one SFR-view screenshot showing `I2C1->SR1` with `AF` set and `SPI2->CR1` with your mode bits — plus your measured line time and the refresh rate you justified from it.

If you are stuck

Scan finds nothing: `SDA`/`SCL` swapped, LCD not powered, or pull-ups missing. **Scan finds a device but writes fail:** the 7-bit/8-bit shift. **Display lit but blank:** contrast trimmer, or the 4-bit init sequence. **Display shows garbage:** nibble order in the init. **SPI returns 0x00:** jumper not seated, or `SPE` not set. **SPI returns shifted data:** wrong `CPOL`/`CPHA`.